WEEK 1

```
listOfTasks = []

def createTask():
    taskName = input("Task Name: ")
    taskDescription = input("Description: ")

    print("Would you like to provide instructions or steps?")
    print("1. Instructions")
    print("2. Steps")
    taskChoice = input("Your choice (type the number): ")

    taskProcedureInstructions = None
    taskProcedureSteps = None

    if taskChoice == "1":
        taskProcedureInstructions = input("Enter your instructions: ")

    elif taskChoice == "2":
        taskProcedureSteps = []
        print("Enter steps (type DONE to finish):")

        while True:
            taskProcedureStep = input("- ")

            if taskProcedureStep.upper() == "DONE":
                break

            taskProcedureSteps.append(taskProcedureStep)
    elif taskChoice == "3":
        print("No instructions or steps will be added.")
        taskProcedureInstructions = None
        taskProcedureSteps = None

    else:
        print("Please enter appropriate input")
```

```
taskTime = input("Time to finish (1:30PM - 2:30PM): ")

print("What would this task be classified as?")
print("1. Daily Task")
print("2. Weekly Task")

taskCategorychoice = input("Your choice (type the number): ")

if taskCategorychoice == "1":
    taskCategory = "Daily Task"
elif taskCategorychoice == "2":
    taskCategory = "Weekly Task"
else:
    print("Please enter appropriate input")
    taskCategory = "Unspecified"

print("Please enter the deadline:")
taskDeadline = input("")
```

WEEK 2

```
newTask = {
    "taskName": taskName,
    "taskDescription": taskDescription,
    "taskProcedureInstructions": taskProcedureInstructions,
    "taskProcedureSteps": taskProcedureSteps,
    "taskTime": taskTime,
    "taskCategory": taskCategory,
    "taskDeadline": taskDeadline
}

return newTask
```

WEEK 3

```
def addTasks(listOfTasks):
    newTask = createTask()
    listOfTasks.append(newTask)
```

WEEK 4

```
def taskDisplay(listOfTasks):
    print("\n||||| Display of Tasks |||||")

    for task in listOfTasks:
        print("\n--------------------------")
        print("Task:", task["taskName"])
        print("Description:", task["taskDescription"])

        if task["taskProcedureInstructions"] is not None:
            print("Procedure:", task["taskProcedureInstructions"])

        if task["taskProcedureSteps"] is not None:
            print("Procedure Steps:")
            for taskProcedureStep in task["taskProcedureSteps"]:
                print("-", taskProcedureStep)

        print("Time Duration:", task["taskTime"])
        print("Category:", task["taskCategory"])
        print("Deadline:", task["taskDeadline"])


while True:
    addTasks(listOfTasks)

    moreTasks = input("Add another task? (y/n): ")

    if moreTasks.lower() == "n":
        break

taskDisplay(listOfTasks)
```

WEEK 5-8

```python
import tkinter as tk
from tkinter import messagebox, ttk
import json
from datetime import datetime

FILE_NAME = "tasks.json"

class TaskManager:
    def __init__(self, root):
        self.root = root
        # Renamed window title as requested
        self.root.title("Task Manager")
        self.root.geometry("600x750")

        self.tasks = self.load_tasks()
        self.timer_running = False
        self.target_time = None

        self.setup_ui()
        self.update_list()

    def load_tasks(self):
        try:
            with open(FILE_NAME, "r") as f:
                content = f.read()
                return json.loads(content) if content else []
        except (FileNotFoundError, json.JSONDecodeError):
            return []

    def save_tasks(self):
        with open(FILE_NAME, "w") as f:
            json.dump(self.tasks, f, indent=4)

    def setup_ui(self):
        # --- Input Section ---
        input_frame = ttk.LabelFrame(self.root, text="Create New Task", padding="10")
# Name
ttk.Label(input_frame, text="Task Name:").grid(row=0, column=0, sticky="w")
self.name_entry = ttk.Entry(input_frame, width=45)
self.name_entry.grid(row=0, column=1, pady=2, padx=5)

# Description
ttk.Label(input_frame, text="Description:").grid(row=1, column=0, sticky="w")
self.desc_entry = ttk.Entry(input_frame, width=45)
self.desc_entry.grid(row=1, column=1, pady=2, padx=5)

# Category
ttk.Label(input_frame, text="Category:").grid(row=2, column=0, sticky="w")
self.category_var = tk.StringVar(value="Daily Task")
categories = ["Daily Task", "Weekly Task", "Work", "Personal"]
self.category_menu = tk.OptionMenu(input_frame, self.category_var, *categories)
self.category_menu.grid(row=2, column=1, sticky="w", pady=2, padx=5)

# Timer Type Selection
self.timer_type = tk.StringVar(value="deadline")
radio_frame = ttk.Frame(input_frame)
radio_frame.grid(row=3, column=0, columnspan=2, pady=5)
ttk.Radiobutton(radio_frame, text="Deadline Countdown", variable=self.timer_type, value="deadline").pack(side=
ttk.Radiobutton(radio_frame, text="Manual Time Range", variable=self.timer_type, value="range").pack(side="lef

# Deadline Input
ttk.Label(input_frame, text="Deadline (YYYY-MM-DD HH:MM:SS):").grid(row=4, column=0, sticky="w")
self.deadline_entry = ttk.Entry(input_frame, width=45)
self.deadline_entry.insert(0, datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
self.deadline_entry.grid(row=4, column=1, pady=2, padx=5)

# Manual Range Input
ttk.Label(input_frame, text="Manual Range (e.g. 9AM-5PM):").grid(row=5, column=0, sticky="w")
self.range_entry = ttk.Entry(input_frame, width=45)
self.range_entry.grid(row=5, column=1, pady=2, padx=5)
```

```
# --- Action Buttons ---
btn_frame = ttk.Frame(self.root)
btn_frame.pack(pady=10)
ttk.Button(btn_frame, text="Add Task", command=self.add_task).pack(side="left", padx=5)
ttk.Button(btn_frame, text="Delete Selected", command=self.delete_task).pack(side="left", padx=5)

# --- Task List Display ---
list_frame = ttk.Frame(self.root, padding="10")
list_frame.pack(fill="both", expand=True)

ttk.Label(list_frame, text="Your Tasks (Click to see description):", font=("Arial", 10, "bold")).pacl
self.task_list = tk.Listbox(list_frame, font=("Arial", 10), height=10)
self.task_list.pack(side="left", fill="both", expand=True)
self.task_list.bind('<<ListboxSelect>>', self.on_task_select)

scroller = ttk.Scrollbar(list_frame, orient="vertical", command=self.task_list.yview)
scroller.pack(side="right", fill="y")
self.task_list.config(yscrollcommand=scroller.set)

# --- Dynamic Description Area ---
self.details_frame = ttk.LabelFrame(self.root, text="Task Description", padding="10")
self.details_frame.pack(fill="x", padx=10, pady=5)

self.desc_display = ttk.Label(self.details_frame, text="Select a task from the list.", wraplength=500
self.desc_display.pack(anchor="w")

# --- Live Timer ---
self.timer_label = tk.Label(self.root, text="00:00:00", font=("Helvetica", 28), fg="blue")
self.timer_label.pack(pady=10)

ttk.Button(self.root, text="Start Countdown", command=self.start_timer).pack(pady=5)

add_task(self):
name = self.name_entry.get()
```

```
    time_info = f"Due: {t['deadline']}" if t['deadline'] else f"Range: {t['time_range']}"
    display_text = f"{t['name']} | {t['category']} | {time_info}"
    self.task_list.insert(tk.END, display_text)

on_task_select(self, event):
selected = self.task_list.curselection()
if selected:
    task = self.tasks[selected[0]]
    desc = task.get("description", "No description provided.")
    if not desc: desc = "No description provided."
    self.desc_display.config(text=desc)

start_timer(self):
selected = self.task_list.curselection()
if not selected:
    messagebox.showwarning("Warning", "Please select a task from the list first.")
    return

task = self.tasks[selected[0]]
if not task.get("deadline"):
    messagebox.showinfo("Timer", "Manual Range tasks do not support live countdowns.")
    return

try:
    self.target_time = datetime.strptime(task["deadline"], "%Y-%m-%d %H:%M:%S")
    self.timer_running = True
    self.tick()
except ValueError:
    messagebox.showerror("Error", "Check your Date Format (YYYY-MM-DD HH:MM:SS)")

tick(self):
if not self.timer_running:
    return
```

```
def tick(self):
    if not self.timer_running:
        return

    diff = self.target_time - datetime.now()

    if diff.total_seconds() <= 0:
        self.timer_label.config(text="Time's up!", fg="red")
        self.timer_running = False
        messagebox.showinfo("Alert", "A task deadline has been reached!")
    else:
        # Format time remaining
        time_str = str(diff).split(".")[0]
        self.timer_label.config(text=time_str, fg="black")
        self.root.after(1000, self.tick)
```

```
if __name__ == "__main__":
    root = tk.Tk()
    app = TaskManager(root)
    root.mainloop()
```